GRAMMAR_ADDENDUM

Grammar Addendum — slash form + namespace prefix + arrow form (operator mandate, ; arrow form)

EXTENDS the § . . action-prefix design with additional equivalences. Every registered action (e.g. BACKGROUND, REMINDER) MUST be invocable in ALL of these EQUIVALENT forms — same action, same expansion, same execution:

Form	Example	Notes
`ACTION_NAME :: <rest>`	`BACKGROUND :: Do X`	bare `::` form (no namespace)
`PRE-FIX::AC-TION_NAME :: <rest>`	`DEFAULT::BACKGROUND :: Do X`	namespaced `::` form
`/ACTION_NAME <rest>`	`/BACKGROUND Do X`	bare slash form — ONLY if `/AC-TION_NAME` does NOT collide with an existing/built-in slash command of the host agent
`/PRE-FIX::ACTION_NAME <rest>`	`/DEFAULT::BACKGROUND Do X`	namespaced slash form — disambigu-ates from any ex-isting `/AC-TION_NAME`; ALWAYS safe
`ACTION_NAME ---> <rest>` (and `PRE-FIX::ACTION_NAME ---> <rest>`)	`BACKGROUND ---> Do X` / `DE-FAULT::BACKGROUND ---> Do X`	arrow form — the `--->` body separ-ator (one ASCII space on each side); a third equivalent delim-iter alongside `::` and `/`

Namespace prefix

- `PREFIX` is an action NAMESPACE. The reserved default namespace is **`DEFAULT`**. Custom namespaces are permitted (e.g. a project/plugin namespace) so the same action can run under the default OR a custom prefix.
- An action is executable WITH or WITHOUT the prefix: `BACKGROUND ::` ≡ `DEFAULT::BACKGROUND ::` ≡ `/BACKGROUND` ≡ `/DEFAULT::BACKGROUND` ≡ `BACKGROUND --->` ≡ `DEFAULT::BACKGROUND --->`.
- Namespace separator inside the token is `::` (no surrounding spaces): `PREFIX::ACTION_NAME`. This is DISTINCT from the action-body separator `::` (spaces both sides) used in forms `/` between the token and `<rest>`, and the arrow-body separator `--->` (spaces both sides) used in form .

Grammar (extended, anchored to first non-blank line)

- `::`-form regex: `^(?:([A-Z][A-Z0-9_]*)::)?([A-Z][A-Z0-9_]*) :: (.*)$` → group =optional PREFIX, group =ACTION_NAME, group =rest.
- slash-form regex: `^/(?:([A-Z][A-Z0-9_]*)::)?([A-Z][A-Z0-9_]*)\s+(.*)$` → group =optional PREFIX, group =ACTION_NAME, group =rest.
- arrow-form regex: `^(?:([A-Z][A-Z0-9_]*)::)?([A-Z][A-Z0-9_]*) ---> (.*)$` → group =optional PREFIX, group =ACTION_NAME, group =rest. The `--->` separator is anchored immediately after the leading token; a mid-prose `--->` after a colon token (e.g. `A :: B ---> C`) parses as the COLON form (the arrow match falls through), so the `::`, arrow, and slash forms never cross-interfere.

- Token case: ACTION_NAME and PREFIX are UPPERCASE `[A-Z][A-Z0-9_]*` (directive/macro convention — DISTINCT from the § . . lowercase file/dir naming, which still governs the registry FILE names + script names). `DEFAULT` is the reserved default-namespace literal.
- Escape: leading `\` makes the line literal (no expansion): `\BACKGROUND :: x`, `\BACKGROUND ---> x`, and `\/BACKGROUND x`.
- Unknown grammar-shaped token (looks like an action but not in the registry) → ask per § . . /§ . . , NEVER silently expand/drop (§ . .).

Conflict rule (slash form)

- `/ACTION_NAME` (form) is honored as the action ONLY when `ACTION_NAME` (case-folded for the collision check) does not match a built-in/host slash command. The registry MAY carry a per-action `slash_bare: true|false|auto` and a `slash_conflicts: [..]` list; `auto` = enable bare slash unless a known host command collides.
- Form (`/PREFIX::ACTION_NAME`) is ALWAYS unambiguous and always honored.

Registry schema additions

```yaml
grammar:
  default_namespace: DEFAULT
  namespace_separator: '::'        # inside the token, no spaces
  body_separator: ' :: '           # token <-> rest, spaces both sides
        ('::' form)
  arrow_body_separator: ' ---> '   # token <-> rest, spaces both sides
        (arrow form)
  slash_prefix: '/'
  slash_body_separator: ' '        # token <-> rest (slash form)
  # regexes: colon_form_regex / slash_form_regex / arrow_form_regex
actions:
  - name: BACKGROUND
    namespaces: [DEFAULT]          # which namespaces this action is
        registered under
    slash_bare: auto               # bare /BACKGROUND honored unless
        host-command collision
    slash_conflicts: []            # known colliding host slash commands
        (forces /DEFAULT::BACKGROUND)
    # ... expansion, rules, composes_with as before
  - name: REMINDER
    namespaces: [DEFAULT]          # second registered action (verify-
        don't-assume status re-surfacing)
    slash_bare: auto
    slash_conflicts: []
    # ... expansion, rules, composes_with
```

Impl impact

- `apx_parse_prefix` MUST recognize all forms, returning (namespace|DEFAULT, action, rest, matched_form ∈ {colon, slash, arrow}). The python and awk parse paths stay byte-identical; the awk arrow branch FALLS THROUGH to the colon check on an invalid leading token so `A :: B ---> C` parses identically in both paths.
- `apx_expand_prompt` resolves namespace+action against the registry; bare-slash honored per the conflict rule; the arrow form resolves to the SAME action/expansion as the `::` and slash forms.
- LAYER- hook handles all first-line forms.

- The slash-command generator already emits `/background` (form) per agent; ADD the namespaced `/default::background` (form) where the agent's command syntax allows `::` (else a `default-background` alias) + document the collision fallback. The arrow form ( ) is a free-form/typed delimiter, not a generated slash command.
- recognition_instruction.md teaches the agent all forms + the conflict rule + escape + the `REMINDER` action.

Naming: action/namespace tokens UPPERCASE; FILES/DIRS lowercase snake_case (§ . .). DEFAULT is the reserved default namespace literal.